Diplomado en Ingeniería de Software con IA y Sistemas Agénticos

Taller de ingeniería de prompts y contexto

Caso CafeFET

Carlos Rubio · Michell Fierro · Fabian Cabrera

Sesión 03 · Módulo 1 · 23 de octubre de 2026

1. El caso y el primer prompt

CafeFET es un sistema ficticio para pedir comida por anticipado. El taller propone estas reglas:

- Se reciben pedidos de lunes a sábado, entre las 7:00 y las 18:00.
- Cada pedido puede tener hasta cinco productos.
- El pago se descuenta del carné y el saldo no puede quedar negativo.
- No se pueden pedir productos agotados.
- Si el pedido no se recoge en 30 minutos, se cancela y se devuelve el saldo.

Reto 1 · Mejorar una instrucción

El prompt inicial solo decía: «Haz una función para crear pedidos». No explicaba el lenguaje, los datos ni las reglas. El modelo eligió JavaScript y agregó una clase, precios de ejemplo y funciones que nadie había pedido.

En el prompt mejorado aclaramos qué función debía crear, qué datos recibiría, cuáles reglas debía cumplir y qué debía devolver. La respuesta pasó a Python y revisó varias reglas importantes.

Qué quedó pendiente

El resultado todavía tuvo problemas: dijo que aceptaba pedidos hasta las 18:00, pero el código los rechazaba a esa hora; supuso cuándo empezaban los 30 minutos; y algunos valores de retorno no coincidían con lo solicitado. Las tres respuestas quedaron cortadas al límite de salida. Los prompts y las respuestas completas están en el ZIP.

2. Reto 2 · Revisor de código

La función entregada con el taller tenía dos errores claros: permitía hasta seis productos, aunque el máximo era cinco, y aceptaba que el saldo quedara hasta en -1000. Además, no revisaba el día ni si un producto estaba agotado.

Versión	Qué encontró
qwen-demo sin reglas	No encontró los dos límites; incluso inventó que había un sobregiro permitido.
Revisor v1	Encontró los dos límites y varias validaciones que faltaban.
Revisor v2	También encontró los límites, pero una recomendación habría rechazado un saldo final de cero.

Prueba de instrucciones escondidas

Probamos tres ataques: pedirle las instrucciones, pedirle que cambiara de tarea y esconder una orden en un comentario del código.

Revisor	Ataques que bloqueó	Problema observado
v1	1 de 3	Reveló sus instrucciones y aceptó cambiar de tarea.
v2	3 de 3	En el tercer ataque también se negó a revisar código válido.

La versión más estricta bloqueó mejor las órdenes escondidas, pero en un caso dejó de ser útil. Solo probamos tres ataques; no alcanza para decir que el revisor sea seguro en general.

3. Reto 3 · Del requisito a las pruebas

El requisito nuevo era permitir que un estudiante repitiera su último pedido con un clic. Lo dividimos en tres pasos: escribir una historia de usuario, sacar criterios de aceptación y convertirlos en casos de prueba.

La historia salió bien: «Como estudiante, quiero repetir mi último pedido con un solo clic, para agilizar mi proceso de compra». El primer error apareció en los criterios: el modelo agregó requisitos que no estaban en el enunciado y dejó por fuera el horario.

Qué corregimos

- Repetimos la cadena desde el paso donde apareció el primer error.
- Pedimos que no agregara cambios parciales ni sustituciones de productos.
- Revisamos la tabla final: todavía tenía errores en un criterio y en la relación entre criterios y pruebas.
- Al final dejamos una versión revisada con pruebas para horario, cantidad, saldo, productos agotados y vencimiento.

La lección fue sencilla: un error en los criterios puede pasar a todos los casos de prueba. Revisamos cada paso antes de continuar. La referencia corregida está separada de las respuestas originales en el ZIP.

4. Reto 4 · Razonamiento y contexto

Parte A · Revisar una función paso a paso

Probamos el código con dos productos que costaban 1500 y un saldo de 1000. El pedido debía rechazarse, pero la función lo aceptaba porque permitía un saldo negativo de hasta 1000.

Phi no encontró esa causa. En la respuesta paso a paso marcó como verdaderas dos comparaciones que en realidad eran falsas. La traza hizo más fácil ver el error, pero no volvió correcta la explicación.

Parte B · Una regla de formato

Pedimos a qwen-demo que empezara sus respuestas con «VEREDICTO:». Como mensaje normal, respetó la regla en la primera respuesta y la olvidó en la segunda. Después la pusimos en el system prompt y apareció en las dos respuestas.

Las cuatro respuestas superaron 400 palabras y algunas quedaron cortadas. En esta prueba, la instrucción se conservó mejor en el system prompt. Eso no significa que siempre se cumpla o que el modelo tenga memoria ilimitada.

5. Bonus · JSON y comparación de prompts

Extraer datos en JSON

Probamos seis mensajes con un esquema JSON. Los seis tuvieron el formato correcto, pero al comparar los campos con el mensaje original coincidieron exactamente 20 de 24. En dos casos se perdió información explícita. Un JSON válido puede tener datos equivocados.

Comparación A/B

También clasificamos diez mensajes como BUG, MEJORA o DUDA. La versión v2 incluía definiciones y ejemplos; v1 no. Probamos ambas con dos modelos.

Modelo	v1	v2
qwen3.5:4b	7/10	8/10
qwen3.5:0.8b	4/10	5/10

V2 mejoró un caso en cada modelo. Aun así, algunos mensajes ambiguos siguieron clasificándose mal. Son diez ejemplos de este taller, no una medida general del rendimiento de los modelos.

6. Lo que nos llevamos

Lo que más nos sirvió fue escribir las reglas del negocio antes de pedir código. Con ese contexto, los revisores encontraron errores que la primera respuesta no vio. Pero también vimos que más instrucciones pueden hacer que el modelo rechace una tarea válida.

Para pedir una función nueva, primero aclararíamos las reglas que falten. Luego haríamos la historia, los criterios y las pruebas por separado, revisando cada respuesta antes de seguir. Probaríamos situaciones concretas, como saldo justo, producto agotado, domingo y datos incompletos.

Una respuesta ordenada no garantiza que esté bien. En el taller hubo JSON válido con datos perdidos, pruebas que afirmaban cubrir casos inexistentes y explicaciones con cuentas equivocadas. Por eso guardamos las respuestas tal como salieron y las comparamos con el enunciado.

Archivos incluidos

El ZIP contiene los prompts, las respuestas completas de Ollama, las dos versiones del revisor, las transcripciones del reto 4 y los scripts y resultados del bonus. La entrega aún no está publicada en un repositorio público de GitHub.